



REPORTE DE INVESTIGACIÓN

MULTITHREADING EN LA
VIDA REAL

SISTEMAS OPERATIVOS
Semestre 2026-2

Fecha de entrega:
04/05/2026

Universidad Nacional
Autónoma de México
Facultad de Ingeniería

ALUMNOS:

TORRES LOZANO LUIS

No. de cuenta: 318209636

Correo: luistorres.lt716@gmail.com

Grupo 07

ZAVALA MAGAÑA LUIS ARTURO

No. de cuenta: 321045182

Correo: luiz.zavala.p8@gmail.com

Grupo 07

DOCENTE:

DR. GUNNAR EYAL WOLF ISZAEVICH

Índice

1. Introducción	2
2. Marco Teórico	2
2.1. ¿Qué es un proceso?	2
2.2. ¿Qué es un hilo o <i>thread</i> ?	3
2.3. Modelos de planificación de hilos	3
3. MT	4
3.1. ¿Cómo funciona?	4
3.2. Objetivo del MT	4
3.3. Multihilo vs. multitarea vs. multiprocesamiento	5
4. Aplicaciones del MT en la vida real	5
4.1. Navegadores web	5
4.2. Aplicaciones móviles	5
4.3. Videojuegos	6
4.4. Servidores web	6
4.5. Sistemas bancarios	6
5. Ventajas y Desventajas	7
6. Ejemplo práctico en C	7
7. Conclusión	9

1. Introducción

En la actualidad, la tecnología forma parte fundamental de la vida cotidiana. Las aplicaciones móviles, navegadores web, videojuegos y sistemas bancarios requieren ejecutar múltiples tareas al mismo tiempo para ofrecer rapidez, eficiencia y una buena experiencia al usuario. Para cumplir con estas necesidades, los desarrolladores utilizan diversas técnicas de programación, entre las cuales destaca el *multithreading* (MT).

El MT es una técnica que permite a un programa dividir su ejecución en varios hilos o tareas independientes que pueden ejecutarse de manera concurrente o paralela. Gracias a ello, es posible realizar varias acciones dentro de una misma aplicación sin que el sistema se vuelva lento o deje de responder. Según Adesanya et al., los programas multihilo pueden utilizar de manera efectiva todos los núcleos disponibles del procesador, lo que resulta en una ejecución más rápida y una mejor utilización de los recursos del sistema [6].

La presente investigación tiene como objetivo explicar qué es el MT, cómo funciona, cuáles son sus principales ventajas y desventajas, y de qué manera se aplica en situaciones reales en aplicaciones de uso cotidiano. Asimismo, se presentará un ejemplo práctico en lenguaje C para comprender mejor su implementación a bajo nivel.

2. Marco Teórico

2.1. ¿Qué es un proceso?

Un proceso es una instancia de un programa en ejecución. Cuando un usuario abre una aplicación, el sistema operativo le asigna recursos como memoria, tiempo de CPU y acceso a archivos para que pueda funcionar. Cada proceso cuenta con su propio espacio de memoria aislado, por lo que si se abren dos aplicaciones distintas, como un navegador y un editor de texto, cada una opera de forma independiente sin interferir con la otra.

Según Cerón, la información de control de un proceso se almacena en el *Bloque de Control del Proceso* (PCB), que reside en el espacio del núcleo del sistema operativo y contiene datos como el identificador del proceso, su prioridad, el estado de los registros del procesador y la tabla de archivos abiertos [2]. El sistema operativo utiliza esta estructura para realizar los cambios de contexto entre procesos.

Ciclo de vida de un proceso

Cerón describe el ciclo de vida estándar de un proceso mediante cinco estados [2]:

- **Creado:** el proceso acaba de ser inicializado por el sistema operativo.
- **Listo:** el proceso está en condiciones de usar la CPU y espera en la cola del planificador a que le sea asignado tiempo de procesador.
- **En ejecución:** el planificador le ha asignado tiempo de CPU y el proceso está ejecutando sus instrucciones.
- **Bloqueado:** el proceso no puede continuar hasta que ocurra un evento externo, como la finalización de una operación de E/S o la señal de otro proceso.

- **Terminado:** el proceso ha completado su ejecución o ha sido finalizado por el sistema operativo.

Este mismo ciclo aplica a los hilos dentro de un proceso, con la diferencia de que el planificador de hilos puede operar tanto a nivel del sistema operativo como a nivel de la aplicación, dependiendo del tipo de hilo.

2.2. ¿Qué es un hilo o *thread*?

Un hilo o *thread* es la unidad más pequeña de ejecución dentro de un proceso. Cerón lo define como cada secuencia de control dentro de un proceso que ejecuta sus instrucciones de forma independiente [2]. Los hilos que pertenecen a un mismo proceso comparten el mismo espacio de memoria y los mismos archivos abiertos, lo que los hace más ligeros que los procesos y permite que se comuniquen entre sí de forma directa sin necesidad de mecanismos de comunicación entre procesos (IPC) [1].

Esta compartición tiene dos consecuencias importantes: los hilos son más rápidos de crear que un proceso nuevo, y el cambio de contexto entre hilos es mucho más eficiente que entre procesos, ya que no requiere intercambiar la información del espacio de memoria, sino únicamente los registros del procesador y el puntero de pila [1]. Benchmarks reales en Linux miden el costo de un cambio de contexto entre hilos en aproximadamente 1.2 a 1.5 microsegundos en hardware moderno, comparado con los varios microsegundos que tarda un cambio de contexto entre procesos completos [11]. Dado que los cambios de contexto pueden producirse miles de veces por segundo, este ahorro se amplifica considerablemente en la práctica.

Existen dos categorías principales de hilos según cómo interactúan con el sistema operativo [1, 2]:

- **Hilos de usuario (*user threads* o *green threads*):** son gestionados completamente por la aplicación o el entorno de ejecución del lenguaje, sin que el sistema operativo los conozca directamente. Son portables entre plataformas, pero presentan una limitación crítica: el sistema operativo los trata como un único hilo de kernel. Por ello, si uno de estos hilos realiza una llamada bloqueante al sistema, todo el proceso se bloquea, y nunca pueden aprovechar el paralelismo real de los procesadores multinúcleo [2].
- **Hilos de kernel (*kernel threads*):** el sistema operativo los gestiona directamente, tratándolos como procesos ligeros (LWP, *Lightweight Processes*). Esto le permite asignarlos a distintos núcleos para ejecutarse de forma verdaderamente paralela, y si uno se bloquea, los demás continúan ejecutándose sin interrupción. Son los que utilizan todas las aplicaciones de alto rendimiento en la práctica: pthreads en C, Java Platform Threads en versiones modernas de la JVM, y los threads de Win32 en Windows [1].

2.3. Modelos de planificación de hilos

Cerón describe tres modelos principales para la relación entre hilos de usuario e hilos de kernel [2]:

- **Muchos a uno:** múltiples hilos de usuario se mapean a un único hilo de kernel. El proceso no puede aprovechar múltiples núcleos y un bloqueo detiene a todos los hilos. Es el modelo más simple pero con más limitaciones.

- **Uno a uno:** cada hilo de usuario tiene su propio hilo de kernel. Permite paralelismo real y bloqueo individual, pero tiene el costo de una llamada al sistema por cada hilo creado. Este es el modelo que usan Linux (pthreads), Windows (Win32) y la JVM moderna.
- **Muchos a muchos:** un conjunto de hilos de usuario se multiplexa en un conjunto igual o menor de hilos de kernel. Combina flexibilidad con rendimiento y es el modelo más sofisticado, usado en sistemas como Solaris.

3. MT

3.1. ¿Cómo funciona?

El MT funciona dividiendo un programa en múltiples hilos de ejecución que el sistema operativo administra y despacha hacia el procesador. Cada hilo representa una porción del trabajo total, y el planificador decide en qué orden y durante cuánto tiempo ejecutar cada uno según su prioridad y disponibilidad.

Cuando hay un solo núcleo disponible, la CPU alterna entre los hilos mediante un mecanismo llamado *quantum* o rodaja de tiempo: cada hilo recibe entre 4 y 15 milisegundos de tiempo de procesador en sistemas modernos, al cabo de los cuales es desalojado y se ejecuta el siguiente. Esta alternancia es tan rápida que genera la ilusión de simultaneidad, conocida como *pseudosimultaneidad* o concurrencia. Si un hilo se bloquea esperando una operación de E/S antes de agotar su *quantum*, el planificador lo pausa de inmediato y ejecuta otro hilo sin desperdiciar tiempo de CPU.

La ejecución verdaderamente paralela, en la que dos hilos avanzan físicamente al mismo tiempo, solo es posible con hilos de kernel, ya que son los únicos que el sistema operativo puede distribuir entre núcleos físicos distintos. Esto mejora el rendimiento porque mientras un hilo espera una respuesta de red o de disco, otro puede seguir procesando.

Sin embargo, la ganancia de rendimiento al agregar más núcleos no es ilimitada. La Ley de Amdahl, formulada por Gene Amdahl en 1967, establece que el *speedup* máximo posible está limitado por la fracción secuencial del programa: si el 10 % del código no puede paralelizarse, el *speedup* máximo con cualquier número de núcleos es de $10\times$, independientemente de cuántos núcleos se agreguen [7]. Investigación posterior muestra que en la práctica el *speedup* real es aún menor, debido a la sincronización entre núcleos y los efectos de la memoria caché compartida [8].

3.2. Objetivo del MT

El objetivo del MT es aprovechar al máximo los recursos del procesador y reducir los tiempos de espera. En un programa de un solo hilo, si una operación tarda en completarse, toda la aplicación queda detenida hasta que termine. Con MT, el sistema puede atender otra tarea mientras la primera sigue esperando, lo que hace que la aplicación sea más ágil y responda mejor al usuario.

Otro beneficio importante es la adaptabilidad a cambios de prioridad: el planificador puede redirigir el procesador hacia una tarea urgente sin esperar a que las demás terminen.

3.3. Multihilo vs. multitarea vs. multiprocesamiento

La programación multihilo se relaciona, pero no es lo mismo, que la multitarea y el multiprocesamiento:

- La **multitarea** es la capacidad del sistema operativo de ejecutar varios programas de forma concurrente, alternando entre ellos. El MT no hace posible la multitarea, que existía décadas antes en sistemas de los años 60, sino que la hace más eficiente al permitir que un mismo programa tenga varias líneas de ejecución internas que comparten recursos sin necesidad de crear procesos separados.
- El **multiprocesamiento** consiste en usar más de un núcleo o procesador físico para ejecutar instrucciones verdaderamente en paralelo. El MT se complementa con esto cuando se usan hilos de kernel: el sistema operativo puede distribuirlos entre los núcleos disponibles, obteniendo aceleración real proporcional al número de núcleos, sujeta al límite establecido por la Ley de Amdahl.

4. Aplicaciones del MT en la vida real

4.1. Navegadores web

Los navegadores modernos como Google Chrome combinan una arquitectura multiproceso con MT dentro de cada proceso [12]. Cada pestaña corre en su propio proceso, lo que garantiza que el fallo de una no afecte a las demás. Dentro de cada proceso de pestaña conviven hilos con responsabilidades distintas: el hilo principal ejecuta el código JavaScript y gestiona la interfaz de usuario, mientras hilos auxiliares gestionan las operaciones de red, el decodificado de imágenes y la composición gráfica.

Esta arquitectura fue introducida en Chrome en 2008 precisamente para aislar fallos y mejorar la seguridad [12]. Una consecuencia directa es que el hilo de composición puede animar transformaciones CSS y permitir el desplazamiento (*scroll*) de forma fluida incluso cuando el hilo principal está ocupado ejecutando JavaScript pesado. Chrome dispone además de un proceso separado para la GPU, lo que permite que el renderizado 3D no bloquee al proceso del navegador principal.

4.2. Aplicaciones móviles

Las aplicaciones móviles son un ejemplo directo de cómo el sistema operativo impone el uso de MT para garantizar la calidad de la experiencia del usuario. En Android, el sistema exige que toda operación de red o acceso a disco se ejecute fuera del hilo principal de la interfaz (*main thread*); de lo contrario, el sistema detecta el bloqueo y finaliza la aplicación.

En Spotify para Android, el hilo principal gestiona la interfaz y la interacción del usuario, mientras hilos auxiliares se encargan del decodificado y la reproducción del audio, así como de la descarga anticipada del siguiente fragmento de la canción. Esta separación garantiza que la música continúe sin interrupciones incluso si la interfaz está procesando otra acción.

4.3. Videojuegos

En los videojuegos modernos, el MT permite distribuir cargas de trabajo costosas entre varios núcleos del procesador. Es habitual que un hilo se encargue del ciclo de renderizado gráfico, otro maneje la simulación física y de colisiones, y un tercero ejecute la lógica de inteligencia artificial. Separar estas responsabilidades evita que una operación costosa retrase la generación de fotogramas y provoque tirones visibles.

La empresa LINE documentó cómo migró el motor de juegos Cocos2d-x de un sistema de un solo hilo a una arquitectura multihilo para separar la simulación de física del ciclo de renderizado [10]. Con la estructura de un solo hilo, el renderizado solo podía comenzar después de que los cálculos de física terminaran completamente. Con la arquitectura multihilo, el hilo de renderizado utiliza los resultados de la actualización de física anterior mientras la nueva se calcula en paralelo, eliminando ese cuello de botella.

Asimismo, Sung et al. propusieron separar las operaciones de CPU en dos hilos paralelos en motores 3D en red: un hilo para operaciones puras de CPU y otro para operaciones relacionadas con la GPU [9]. Sus experimentos confirmaron que este método permite maximizar el paralelismo en la comunicación entre CPU y GPU, resultando en un renderizado más rápido en sistemas reales.

4.4. Servidores web

Los servidores web como Apache HTTP Server utilizan un modelo de *thread pool*: al iniciarse, crean un conjunto de hilos en espera. Cuando llega una solicitud HTTP de un cliente, se asigna a uno de los hilos disponibles, que la procesa de principio a fin y luego regresa al pool para atender la siguiente. De este modo, el servidor puede atender tantas solicitudes simultáneas como hilos tenga el pool, sin el costo de crear y destruir un hilo por cada petición.

El módulo `mpm.worker` de Apache define los parámetros del pool mediante directivas como `ThreadsPerChild` y `MaxRequestWorkers`, permitiendo ajustar la capacidad del servidor según la carga esperada. Este modelo tiene un límite natural: si todas las conexiones del pool están ocupadas, las nuevas solicitudes se encolan o se rechazan, lo que hace que el dimensionamiento del pool sea una decisión de ingeniería importante.

4.5. Sistemas bancarios

Los sistemas de procesamiento transaccional bancario emplean MT principalmente para atender conexiones concurrentes. Cada sesión activa, ya sea un cajero automático consultando saldo o un cliente realizando una transferencia desde la aplicación móvil, se maneja a través de un hilo del servidor, permitiendo que miles de transacciones se procesen simultáneamente.

Los mecanismos de sincronización son críticos en este contexto. Sin control de concurrencia, si dos hilos leen el saldo de una cuenta al mismo tiempo, ambos ven el mismo valor inicial; luego cada uno calcula su resultado y lo escribe de forma independiente, produciendo un saldo final incorrecto. Por ello, las operaciones sobre saldos requieren mecanismos de exclusión mutua (*mutex*) que garanticen que solo un hilo a la vez pueda modificar una cuenta. En la práctica, esto se complementa con transacciones atómicas en la base de datos para garantizar la consistencia incluso ante fallos del sistema.

5. Ventajas y Desventajas

La siguiente tabla presenta ventajas y desventajas del MT de forma pareada: cada fila relaciona un beneficio con la contraparte o riesgo que surge al intentar obtenerlo.

Ventaja	Desventaja asociada
Mejor uso del procesador: mientras un hilo espera una operación de E/S, otro puede continuar ejecutándose, reduciendo el tiempo ocioso de la CPU.	Mayor complejidad: coordinar el acceso de varios hilos a recursos compartidos requiere sincronización explícita, lo que aumenta la dificultad de diseño y depuración.
Interfaz siempre activa: las tareas pesadas se delegan a hilos en segundo plano, permitiendo que el hilo principal siga respondiendo al usuario sin bloqueos.	Condición de carrera (<i>race condition</i>): si dos hilos acceden y modifican un dato compartido sin sincronización, el resultado puede depender del orden de ejecución, que es no determinista. Los errores de este tipo son especialmente difíciles de reproducir porque pueden aparecer solo bajo carga alta.
Menor costo de cambio de contexto: compartir el espacio de memoria hace que alternar entre hilos sea mucho más eficiente que entre procesos completos, con un costo medido en 1.2 a 1.5 microsegundos en Linux moderno.	Bloqueo mutuo (<i>deadlock</i>): dos hilos pueden quedar bloqueados indefinidamente si cada uno retiene un recurso que el otro necesita para continuar. El programa se congela sin lanzar ningún error visible al usuario.
Paralelismo real en sistemas multinúcleo: con hilos de kernel, el sistema operativo puede asignar cada hilo a un núcleo distinto, logrando ejecución verdaderamente simultánea, sujeta al límite de la Ley de Amdahl.	Inanición (<i>starvation</i>): si las prioridades no están bien definidas, un hilo de baja prioridad puede esperar indefinidamente mientras otros de mayor prioridad acaparan los recursos. La solución habitual es el <i>aging</i> : incrementar gradualmente la prioridad de los hilos que llevan mucho tiempo esperando.

Cuadro 1: Ventajas y desventajas del MT, organizadas por aspecto relacionado

6. Ejemplo práctico en C

Para ilustrar el MT a nivel de sistema, se presenta un programa en C que implementa el patrón productor-consumidor utilizando la biblioteca POSIX Threads (pthreads), estándar en sistemas Linux y macOS [13]. El programa simula un reproductor de música continuo: el hilo reproductor (*consumidor*) toma canciones de una cola compartida y las reproduce una

tras otra, mientras el hilo descargador (*producer*) obtiene las siguientes y las deposita en esa misma cola.

A diferencia de Java, que ofrece estructuras de datos concurrentes como `BlockingQueue` en su biblioteca estándar [3], en C es necesario construir el mecanismo de sincronización de forma explícita. La solución requiere tres componentes: un arreglo circular que actúa como cola, un *mutex* (`pthread_mutex_t`) que protege los índices de la cola para evitar condiciones de carrera, y dos semáforos POSIX (`sem_t`) que implementan el bloqueo inteligente entre productor y consumidor.

El semáforo `llenas` cuenta cuántas canciones hay disponibles en la cola: si su valor es cero, el reproductor se bloquea en `sem_wait(&llenas)` sin consumir CPU hasta que el descargador deposite una nueva canción. El semáforo `vacias` cuenta cuántos espacios libres hay: si la cola está llena, el descargador se bloquea en `sem_wait(&vacias)` hasta que el reproductor consuma una entrada. Este mecanismo evita la espera activa (*busy waiting*) y es el patrón estándar de sincronización entre productores y consumidores [5].

```
#include <pthread.h>
#include <semaphore.h>
#include <string.h>
#include <stdio.h>
#include <unistd.h>

char *cola[4];
int ini = 0, fin = 0;
pthread_mutex_t mtx = PTHREAD_MUTEX_INITIALIZER;
sem_t llenas, vacias; // Semáforos POSIX

void *reproductor(void *arg) {
    char *c;
    while (1) {
        sem_wait(&llenas); // espera si cola vacia
        pthread_mutex_lock(&mtx);
        c = cola[ini++ % 4];
        pthread_mutex_unlock(&mtx);
        sem_post(&vacias); // libera espacio
        if (!strcmp(c, "FIN")) break;
        printf("[Reproductor] Reproduciendo: %s\n", c);
        sleep(3); // simula duracion
    }
    return NULL;
}

void *descargador(void *arg) {
    char *lista[] = {"Cancion B", "Cancion C", "Cancion D", "FIN"};
    for (int i = 0; i < 4; i++) {
        printf("[Descargador] Descargando: %s\n", lista[i]);
        sleep(2); // simula descarga
        sem_wait(&vacias); pthread_mutex_lock(&mtx);
    }
}
```

```

        cola[fin++ % 4] = lista[i];
        pthread_mutex_unlock(&mtx); sem_post(&llenas);
    }
    return NULL;
}

int main() {
    sem_init(&llenas, 0, 0);    // cola empieza vacia
    sem_init(&vacias, 0, 4);    // 4 espacios libres

    cola[fin++ % 4] = "Cancion A"; // ya disponible
    sem_post(&llenas);

    pthread_t hr, hd;
    pthread_create(&hr, NULL, reproductor, NULL);
    pthread_create(&hd, NULL, descargador, NULL);
    pthread_join(hr, NULL);
    pthread_join(hd, NULL);

    sem_destroy(&llenas);
    sem_destroy(&vacias);
    printf("[Main] Reproduccion finalizada.\n");
    return 0;
}

```

Al ejecutar el programa, el reproductor comienza inmediatamente con la canción A (ya disponible en la cola) mientras el descargador trabaja en paralelo. Cuando termina la canción A, la B ya estará descargada esperando en la cola, o bien el reproductor se bloqueará brevemente hasta que el descargador la deposite. Este comportamiento refleja el funcionamiento real de las aplicaciones de *streaming*: la reproducción avanza sin interrupciones porque siempre hay contenido preparado con antelación. Para compilar y ejecutar el programa: `gcc -o reproductor reproductor.c -lpthread && ./reproductor`.

7. Conclusión

El MT es una técnica fundamental en el desarrollo de software moderno que permite ejecutar múltiples tareas dentro de una misma aplicación, mejorando el rendimiento y optimizando el uso de los recursos. Su presencia es transversal a prácticamente todos los sistemas de uso diario: navegadores web, aplicaciones móviles, videojuegos, servidores y sistemas bancarios, como se ha documentado a lo largo de este trabajo con ejemplos concretos de Chrome, Spotify, Apache, motores de juegos y sistemas transaccionales.

El análisis de cada caso muestra un patrón consistente: en todos ellos, el MT se utiliza para separar tareas de distinta naturaleza, especialmente operaciones de E/S y procesamiento, de modo que ninguna bloquee a las demás. Esta separación es la razón por la que las aplicaciones modernas pueden responder al usuario en todo momento, incluso mientras realizan operaciones costosas en segundo plano.

Sin embargo, las ventajas del MT no vienen sin costo. La compartición de memoria entre hilos, que es su principal fortaleza, es también la fuente de sus problemas más difíciles: condiciones de carrera, bloqueos mutuos e inanición requieren un diseño cuidadoso y el uso correcto de mecanismos de sincronización como *mutex* y semáforos. Adicionalmente, la Ley de Amdahl establece un límite teórico al beneficio que se puede obtener del paralelismo, recordando que la fracción secuencial del programa siempre acotará el *speedup* máximo posible [7]. Comprender tanto los beneficios como los riesgos y límites del MT es indispensable para aprovecharlo de forma efectiva en el desarrollo de software actual.

Referencias

- [1] Wolf, G., Ruiz, E., Bergero, F., y Meza, E. (2015). *Fundamentos de Sistemas Operativos* (1.^a ed.). Universidad Nacional Autónoma de México, Instituto de Investigaciones Económicas / Facultad de Ingeniería. Disponible en: <https://sistop.org>
- [2] Cerón, M.E.C. (s.f.). *Programación Concurrente, Capítulo 2: Procesos vs. Hilos*. Benemérita Universidad Autónoma de Puebla.
- [3] Goetz, B., Peierls, T., Bloch, J., Bowbeer, J., Holmes, D., y Lea, D. (2006). *Java Concurrency in Practice*. Addison-Wesley.
- [4] Butcher, P. (2014). *Seven Concurrency Models in Seven Weeks: When Threads Unravel*. The Pragmatic Bookshelf.
- [5] Lea, D. (1999). *Concurrent Programming in Java: Design Principles and Patterns* (2.^a ed.). Addison-Wesley.
- [6] Adesanya, O. et al. (2023). *Introducing Multi-Threaded Programming in Parallel Programming Process for Optimal Performance Results*. ResearchGate. Disponible en: <https://www.researchgate.net/publication/378572252>
- [7] Hill, M.D. y Marty, M.R. (2008). Amdahl's Law in the Multicore Era. *IEEE Computer*. Disponible en: https://research.cs.wisc.edu/multifacet/papers/ieeecomputer08_amdahl_multicore.pdf
- [8] Yavits, L., Morad, A., y Ginosar, R. (2013). *The Effect of Communication and Synchronization on Amdahl's Law in Multicore Systems*. arXiv:1306.3302. Disponible en: <https://arxiv.org/pdf/1306.3302>
- [9] Sung, M.Y. et al. (2006). Parallel Processing for Reducing the Bottleneck in Realtime Graphics Rendering. En: *Advances in Multimedia Information Processing – PCM 2006*. Lecture Notes in Computer Science, vol. 4261. Springer. Disponible en: https://link.springer.com/chapter/10.1007/11922162_107
- [10] LINE Engineering. (2016). *Multi-Threaded Parallel Processing for Physics Simulation in Cocos2d-x*. LINE Engineering Blog. Disponible en: <https://engineering.linecorp.com/en/blog/multi-threaded-parallel-processing-for-physics-simulation-in-cocos2d-x>

- [11] Orlov, E. (2023). *Measuring Context Switching and Memory Overheads for Linux Threads*. Disponible en: <https://eorlov.org/posts/2023/measuring-context-switching-and-memory-overheads-for-linux-threads>
- [12] webperf.tips. (2022). *Multi-process on the Web: The Browser Process Model*. Disponible en: <https://webperf.tips/tip/browser-process-model/>
- [13] Oracle Corporation. (2024). *Java Platform SE: Class Thread y BlockingQueue*. Recuperado de: <https://docs.oracle.com/en/java/javase/21/docs/api/>